



Politecnico di Torino
Team ICARUS PoliTO



Relazione del lavoro svolto in team

Progettazione e realizzazione in laboratorio

Area IT (Divisione Firmware)

Luca Pittalis

12 maggio 2026

Indice

1	Introduzione	1
2	L'ingresso nel team, i primi progetti e i lavori successivi	3
3	Il progetto di DroneCAN	6
3.1	Introduzione	6
3.2	La configurazione fisica	6
3.2.1	Il transceiver	6
3.2.2	L'autopilota	7
3.2.3	Mission Planner	8
3.3	La configurazione di CubeMX	10
3.3.1	FDCAN	10
3.3.2	GPIO & NVIC	11
3.3.3	FreeRTOS	11
3.3.4	Visione generale della configurazione	12
3.4	Il codice	13
3.4.1	Struttura di base	13
3.4.2	Configurazione della comunicazione	13
3.4.3	La gestione dei dati generici del nodo	17
3.4.4	La gestione dei dati della PDU	18
3.4.5	La gestione dei dati dell'MPPT	19
3.4.6	Lo script Python per la visualizzazione dell'MPPT su Mission Planner	21
3.5	La visualizzazione finale	22
4	Conclusione	23
4.1	Resoconto dell'esperienza	23
4.2	Ringraziamenti	24

CAPITOLO 1

Introduzione

Il team Icarus PoliTO consta di 3 progetti in ambito aerospaziale: RA, ACC e APA.

Record Aircraft (RA), nato nel 2017, è un aeromodello il cui obiettivo è parte dell'attuale sviluppo industriale legato alle nuove fonti di energia: questo UAV (unmanned aerial vehicle) si basa sull'energia solare che alimenta il motore elettrico e gli altri sistemi di bordo.

Air Cargo Challenge (ACC) è un UAV cargo capace di portare a termine una specifica missione. Alcuni parametri richiesti per la competizione includono il tipo di payload da trasportare, la velocità con cui questi vengono trasportati e la loro efficienza.

Advanced Propulsion Aircraft (APA) è un nuovo progetto, ancora in via di sviluppo, nato dall'unione dei campi dell'aeronautica e della propulsione a razzo. L'obiettivo è sviluppare un UAV capace di distinguersi per la velocità massima che è in grado di raggiungere, grazie a un motore a razzo appositamente progettato.

Personalmente, ho avuto la possibilità di lavorare come membro dell'area IT, il cui scopo è realizzare tutti i sistemi elettronici e informatici per portare a termine le missioni dei 3 progetti, e più nello specifico nell'area Firmware, operante nella programmazione a basso livello al fine di mettere in comunicazione l'elettronica e i sistemi di controllo delle nostre missioni. Il team è propenso a utilizzare design custom ovunque sia possibile, inclusi l'elettronica, i computer di volo automatici e i sistemi di acquisizione dati.



Figura 1.1: Render di RA



Figura 1.2: ACC in fase di atterraggio

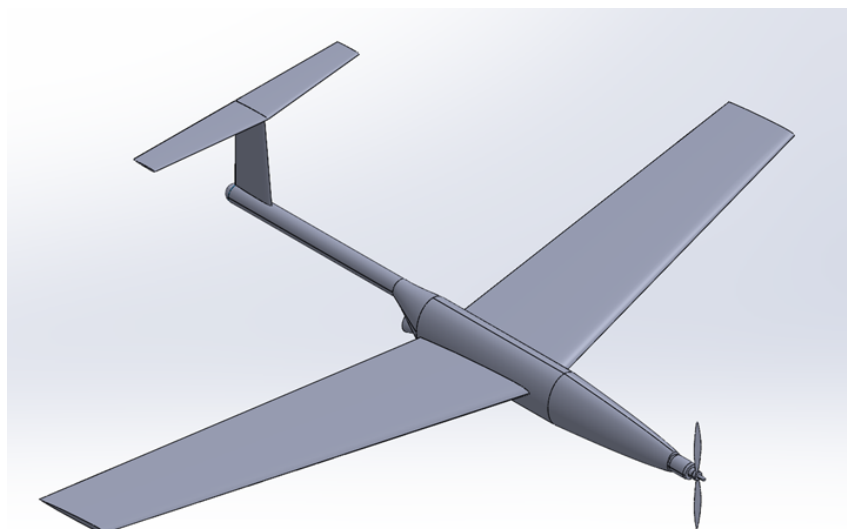


Figura 1.3: Render di APA

CAPITOLO 2

L'ingresso nel team, i primi progetti e i lavori successivi

All'inizio dell'anno accademico 2024-2025 sono entrato in Icarus come membro dell'area IT. Per accedervi sono state necessarie ricerche personali e le conoscenze acquisite nei corsi di "Algoritmi e strutture dati", "Calcolatori elettronici" e "Elettromagnetismo e Teoria dei circuiti" frequentati l'anno precedente. Nei primi incontri settimanali, i membri più anziani del team hanno tenuto diverse lezioni, riguardanti vari ambiti fondamentali per il corretto sviluppo del custom firmware di Icarus.

In particolare, gli argomenti considerati hanno riguardato:

- Utilizzo di Git & GitHub per la gestione delle repository di codice condiviso
- Stabilimento delle convenzioni di codice allo scopo di avere una scrittura in C pulita e ben leggibile
- Utilizzo di CubeMX per la configurazione di microcontrollori STM32
- Comprensione generale dell'utilizzo dei protocolli UART e I2C, utilizzati per la comunicazione tra microcontrollori e periferiche

Dopo qualche sessione introduttiva sulla programmazione dei microcontrollori, abbiamo lavorato a diversi progetti di Rebasing per il controllo e la pulizia del codice di diversi progetti, e in particolare io mi sono dedicato alla verifica di Hyppo, una scheda per comunicazioni wireless con protocollo LoRa atta ad accendere un fuso, così da consentire di innestare la reazione per la partenza di un booster allo stato solido da distanza di sicurezza.

In seguito alla verifica, la scheda è stata attivamente utilizzata in uno static fire per il progetto Dart, eseguito con successo il 16/12/2024. Nello stesso test sono anche state provate, per conto dell'Agenzia Spaziale Europea, delle piastrelle di regolite simil-lunare, al fine di verificare il potenziale impatto di un getto a razzo per l'atterraggio sulla superficie del satellite.

Successivamente alla fase di integrazione nel team, in maniera più indipendente ho avuto la possibilità di collaborare al firmware di Air Telemetry V2, una scheda dedita a raccogliere la telemetria dei vari UAV di Icarus, partecipando poi da zero alla scrittura del codice per l'inizializzazione e la corretta ricezione dei dati di un tubo di pitot. L'obiettivo principale di questo sensore è quello di prendere una temperatura (in °C) e una pressione differenziale (in psi, poi tradotti in Pascal) al fine di calcolare la velocità di un velivolo in un fluido laminare.

Oltre a questo lavoro, mi sono poi occupato anche della formazione dei nuovi membri del team, fattore vitale per consentire al gruppo di continuare a costruire sulle spalle degli studenti che l'hanno preceduto.

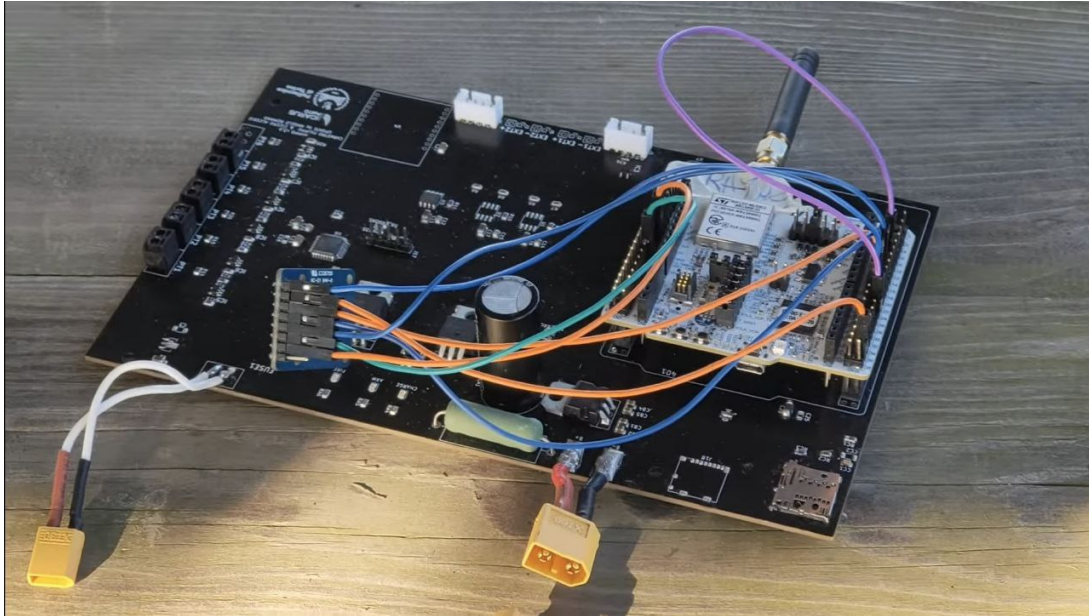


Figura 2.1: Hyppo Board



Figura 2.2: Vista laterale del booster e delle telecamere termiche dell'ESA durante il test statico

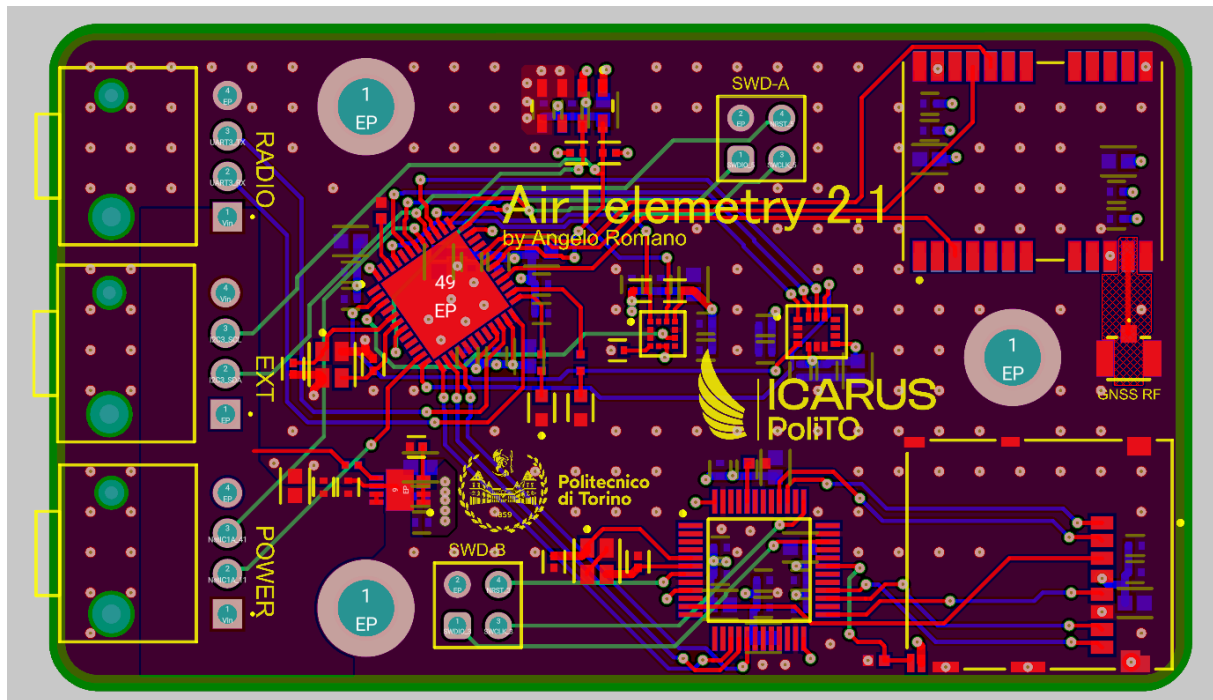


Figura 2.3: Air Telemetry V2 Board

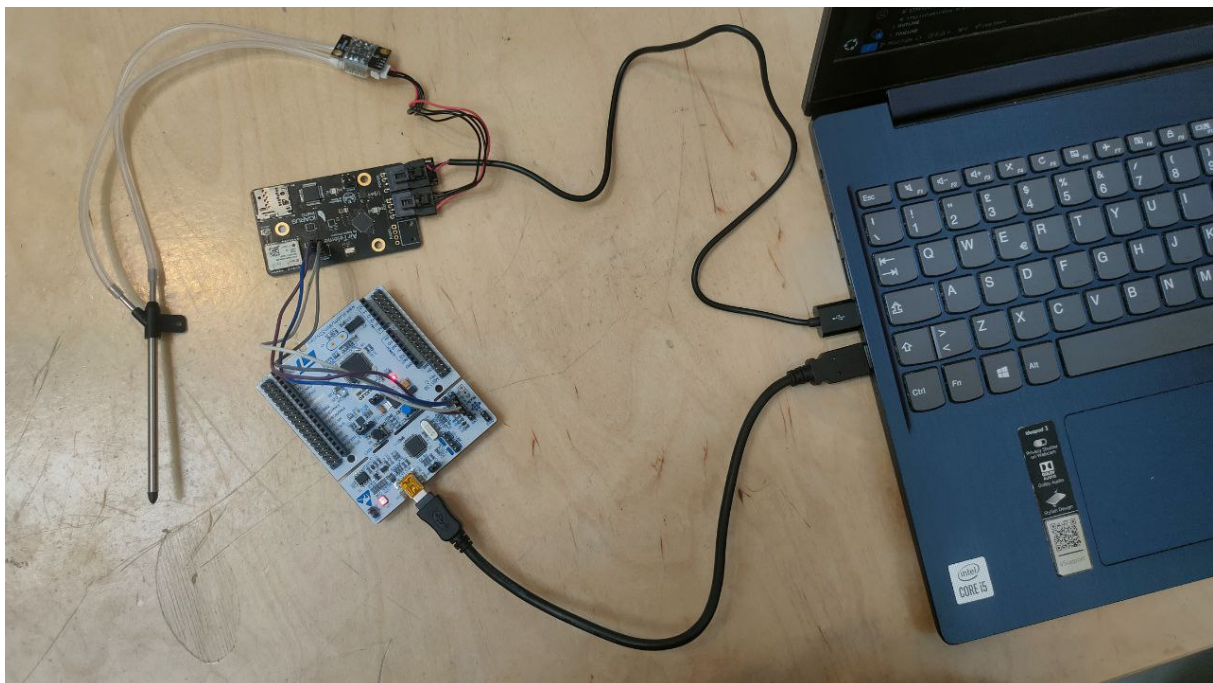


Figura 2.4: Configurazione per il debugging del tubo di pitot tramite Nucleo Board

CAPITOLO 3

Il progetto di DroneCAN

3.1 Introduzione

Il progetto relativo alla configurazione di DroneCAN ha l'obiettivo di creare un'interfaccia diretta di comunicazione tra tre dispositivi principali, ossia:

- Mission Planner, software di ground control station (GCS) usato con il progetto ArduPilot per controllare e configurare droni e veicoli autonomi
- Pixhawk 6C Mini, flight controller (autopilota) per droni e veicoli autonomi
- Scheda Nucleo STM32 (STM32G491RE), microcontrollore per lo sviluppo e la prototipazione di sistemi embedded (soluzione temporanea che verrà sostituita in futuro dalle schede custom firmware di Icarus, RA_PDU e RA_MPPT)

Lo scopo finale è consentire al software di Mission Planner (comunemente configurabile con facilità su qualsiasi PC Windows) di visualizzare in modo efficiente i dati trasmessi dall'autopilota e dalla scheda STM32, così da fornire al team un'interfaccia grafica e di controllo dell'aereo. Per la comunicazione è stato utilizzato il protocollo CAN, nativamente supportato da Mission Planner e dall'autopilota, poi adattato alla scheda tramite transceiver.

NB: Mission planner è tecnicamente disponibile e configurabile anche per Linux e macOS, ma in maniera non nativa e che in questa sede non è stata testata.

3.2 La configurazione fisica

3.2.1 Il transceiver

Per consentire un'adeguata comunicazione tra i dispositivi utilizzati, è stato necessario l'impiego separato di un transceiver della Infineon Technologies, il modello TLE9351BVSJ, grazie al quale è stato possibile convertire i segnali logici del microcontrollore (TX/RX) nei livelli differenziali del bus (CAN_H/CAN_L), così da permettere alla scheda STM32 di inviare i propri dati direttamente all'autopilota, e conseguentemente anche a Mission Planner.

Come denotabile dalla Figura 3.5, il transceiver si presenta come fisicamente separato dalla scheda STM32 utilizzata, in una configurazione che però è solamente temporanea, in quanto le schede finali RA_PDU e RA_MPPT presenteranno già nativamente un transceiver analogo, direttamente collegabile all'autopilota, il quale poi consentirà ai segnali di essere visibili anche a Mission Planner.

3.2.3 Mission Planner

All'interno di Mission Planner, la gestione dei dati prevede che ci siano degli identificativi univoci per ogni nodo, equivalente al dispositivo riconosciuto. Per ognuno di essi possono poi essere riconosciuti numerosi pacchetti, variabili sulla base del nodo.

Come identificativo del singolo dispositivo, tra i vari pacchetti ne sono in realtà sempre richiesti due nello specifico, ossia:

- `uavcan.protocol.NodeStatus`, contenente:
 - Uptime del nodo
 - Health status (OK, warning, error, critical)
 - Mode (operational, initialization, ecc.)
- `uavcan.protocol.GetNodeInfo` (`_req` oppure `_res`), contenente:
 - Node ID
 - Nome del software (node name)
 - Versione hardware/software
 - UID univoco del dispositivo

Entrambi necessari per fare in modo che Mission Planner possa correttamente riconoscere i dati forniti nei pacchetti, assegnandoli (se codificati correttamente) ai rispettivi campi della sua interfaccia interna, poi assegnabili anche nella sezione **Quick** per una visualizzazione più rapida ed intuitiva.

Per verificare che mission planner stia ricevendo correttamente i pacchetti, è sufficiente visualizzare il traffico CAN tramite UAVCAN Inspector, visualizzabile nelle opzioni con il seguente percorso:

Setup → Optional Hardware → DroneCAN/UAVCAN → MAVLinkCAN1 → Connect → Inspector

L'esito dell'ispezione restituisce inizialmente il seguente risultato:

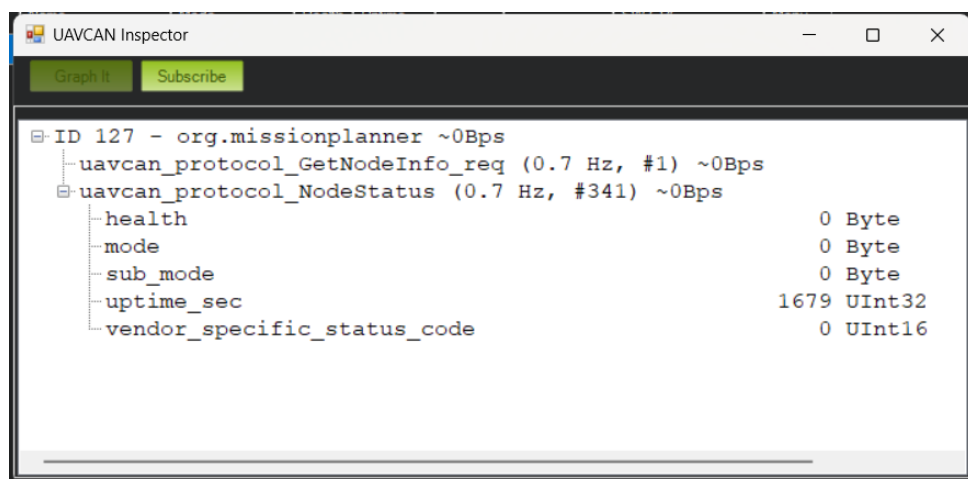


Figura 3.3: Visualizzazione di UAVCAN Inspector senza l'autopilota collegato (un solo ID presente)

Per la configurazione all'interno di Mission Planner, è poi necessario modificare alcuni parametri dell'applicazione al seguente percorso:

Config → Full Parameter List

Per poi salvarli nella memoria dell'autopilota e riavviare la comunicazione (non solamente tramite Mission Planner, ma fisicamente scollegando e ricollegando il dispositivo).

I valori da inserire sono i seguenti:

- CAN_D1_PROTOCOL: 1 (DroneCAN)
- CAN_D1_UC_NODE: 20
- CAN_D1_UC_NTF_RT: 25
- CAN_D1_UC_POOL: 25
- CAN_P1_BITRATE: 500000
- CAN_P1_DRIVER: 1

E nello specifico caso della PDU anche i seguenti:

- BATT_MONITOR: 8 (DroneCAN-BatteryInfo)
- BATT_SERIAL_NUM: -1

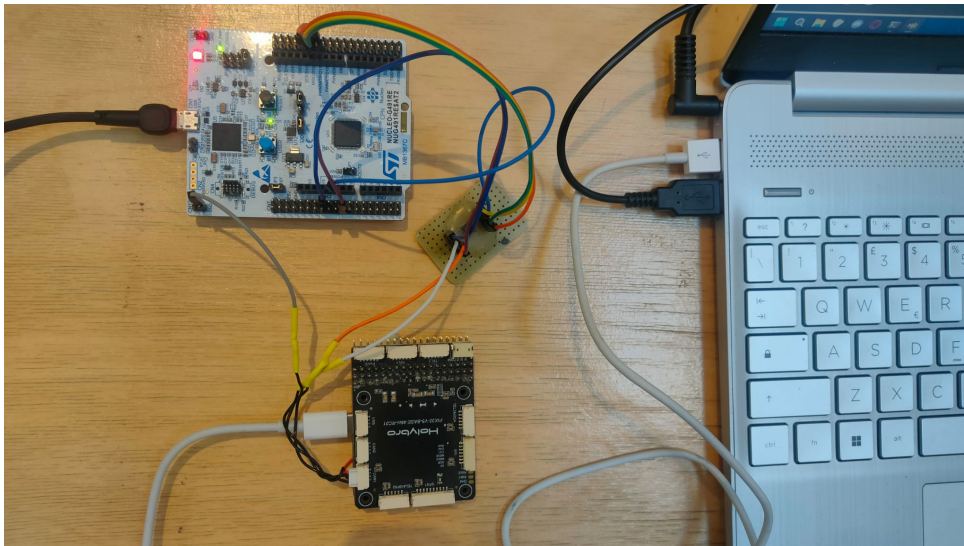


Figura 3.4: Visione d'insieme dei collegamenti di Flight Controller, Nucleo Board e transceiver

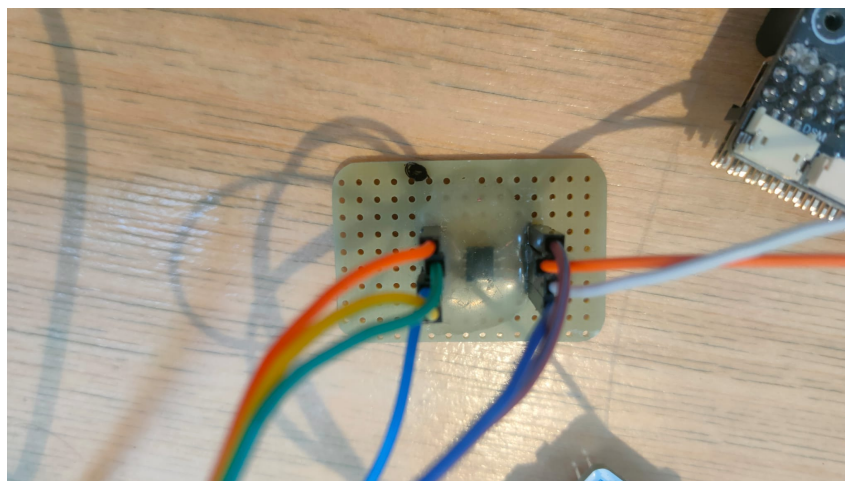


Figura 3.5: Dettaglio del transceiver come componente separato

3.3 La configurazione di CubeMX

La configurazione di CubeMX richiede l'inserimento di numerosi parametri, qui di seguito divisi per categoria e descritti brevemente.

3.3.1 FDCAN

Parametri FDCAN1:

- FDCAN1_TX: Impiegato per la trasmissione sul bus CAN.
Configurazione:
 - Pin: PA12
 - Pin Type: FDCAN1_TX
 - GPIO Pull-up/Pull-down: No pull-up and no pull-down
- FDCAN1_RX: Impiegato per la ricezione sul bus CAN.
Configurazione:
 - Pin: PA11
 - Pin Type: FDCAN1_RX
 - GPIO Pull-up/Pull-down: No pull-up and no pull-down

Parametri generali del protocollo CAN:

- Clock Divider: Divide kernel clock by 1
- Frame Format: Classic mode – Seleziona il formato classico CAN (non CAN FD)
- Mode: Normal mode – Modalità operativa standard del nodo CAN
- Auto Retransmission: Enable – Ritrasmette automaticamente i messaggi non confermati
- Transmit Pause: Disable – Nessuna pausa tra trasmissioni consecutive
- Transmit Exception: Disable – Disabilita gestione speciale errori di trasmissione
- Nominal Sync Jump Width: 13 – Massima correzione della sincronizzazione nel bit timing nominale
- Data Prescaler: 25 – Fattore di divisione del clock per il data bit rate
- Data Sync Jump Width: 1 – Correzione massima della sincronizzazione nella fase dati
- Data Time Seg1: 2 – Durata del primo segmento temporale nella fase dati
- Data Time Seg2: 1 – Durata del secondo segmento temporale nella fase dati
- Std Filters Nbr: 1 – Numero di filtri per ID standard configurati
- Ext Filters Nbr: 0 – Numero di filtri per ID estesi configurati
- Tx Fifo Queue Mode: FIFO mode – Modalità FIFO per la coda di trasmissione
- Nominal Prescaler: 1 – Fattore di divisione del clock per il bit rate nominale
- Nominal Time Seg1: 86 – Durata del primo segmento temporale nominale
- Nominal Time Seg2: 13 – Durata del secondo segmento temporale nominale

3.3.2 GPIO & NVIC

Parametri GPIO:

- LD2: Impiegato dall'MPPT per simulare un input di armamento e disarmamento.
Configurazione:
 - Pin: PA5
 - Pin Type: GPIO_Output,
 - GPIO Pull-up/Pull-down: No pull-up and no pull-down

Parametri NVIC:

- EXTI line[15:10] interrupts: enabled. Attivato per consentire gli interrupt da GPIO_Output.
- FDCAN1 interrupt 0: enabled. Attivato per consentire gli interrupt da FDCAN1.
- FDCAN1 interrupt 1: enabled. Attivato per consentire gli interrupt da FDCAN1.

3.3.3 FreeRTOS

Timers and Semaphores:

- tx_timer: Impiegato per impostare periodicamente FLAG_CAN_PERIODIC.
Configurazione:
 - Callback: tx_timer_callback
 - Type: osTimerPeriodic
 - Code Generation Output: Default
 - Parameter: NULL
 - Allocation: Dynamic

Events:

- rxCANTask: Evento impiegato per la gestione della ricezione di pacchetti CAN.
Configurazione:
 - Allocation: Dynamic
 - Control Block Name: NULL
- txCanTask: Evento impiegato per la gestione dell'invio di pacchetti CAN.
Configurazione:
 - Allocation: Dynamic
 - Control Block Name: NULL

Tasks and Queues:

- receiveCAN: Task per la gestione della ricezione di pacchetti CAN.
Configurazione:
 - Priority: osPriorityNormal
 - Stack Size (Words): 128
 - Entry function: receiveCANFunc
 - Code Generation Option: Default
 - Allocation: Dynamic

- sendCAN: Task per la gestione dell'invio di pacchetti CAN.

Configurazione:

- Priority: osPriorityNormal
- Stack Size (Words): 128
- Entry function: receiveCANFunc
- Code Generation Option: Default
- Allocation: Dynamic

3.3.4 Visione generale della configurazione

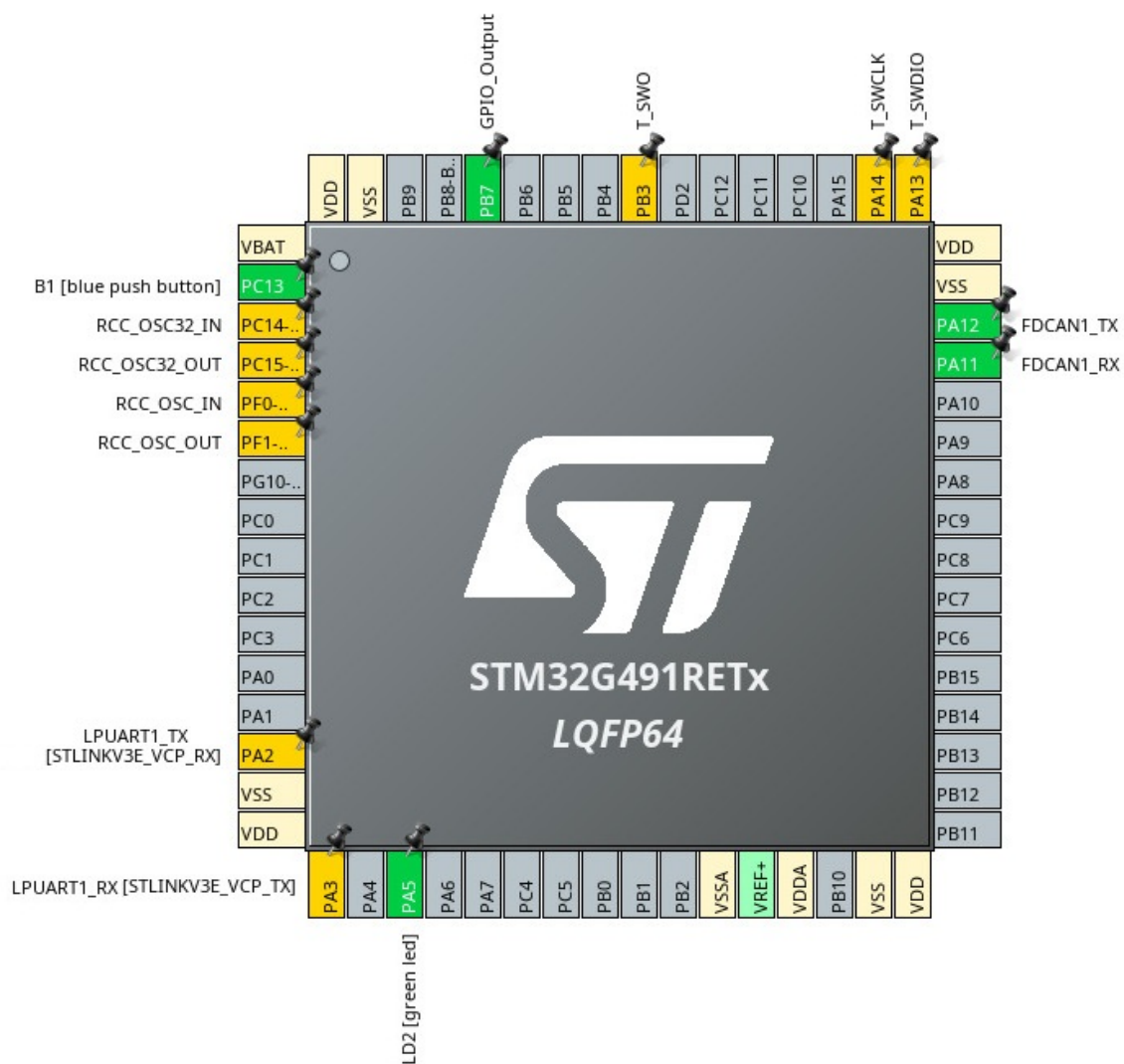


Figura 3.6: Schematica della nucleo board STM32 utilizzata

3.4 Il codice

3.4.1 Struttura di base

Il codice ha lo scopo primario di gestire il flusso di dati generato dalla Nucleo Board, la quale lo codificherà secondo il protocollo DroneCAN e lo invierà all'autopilota, così che questo possa inoltrare tutte le informazioni ricevute anche a Mission Planner. Il software potrà poi a sua volta comunicare con la Nucleo Board secondo il processo inverso, con scopi attualmente solo dimostrativi e non strettamente pratici.

La descrizione sottostante darà per scontata una corretta inizializzazione di CubeMX e delle impostazioni di FreeRTOS ad esso associate, secondo le modalità indicate nel Capitolo 3.3. Inoltre, in questo caso verrà mostrata una versione del firmware operativa contemporaneamente sia per la PDU che per l'MPPT, sebbene in un contesto reale si potrebbe decidere di operare alternativamente anche uno solo dei due dispositivi.

Quest'ultimo passaggio è configurabile facilmente nel file `main.h`, grazie ai seguenti parametri:

```
1 #define ENABLE_BATTERY 1
2 #define ENABLE_MPPT 1
```

3.4.2 Configurazione della comunicazione

Dentro `main.c`, vengono chiamate le seguenti istruzioni di inizializzazione:

```
1 CanardInstance canard;
2 uint8_t memory_pool[2048];
3 FDCAN_RxHeaderTypeDef RxHeader;
4 uint8_t RxData[8];
```

Seguite dall'inizializzazione di periferica FDCAN e canard:

```
1 canardInit(&canard,
2           memory_pool,
3           sizeof(memory_pool),
4           onTransferReceived,           // RX callback
5           shouldAcceptTransfer,        // Acceptance filter callback
6           NULL);
7
8 canardSetLocalNodeID(&canard, CANARD_NODE_ID); // Set UAVCAN node ID
9
10 FDCAN_Config();                               // Configure STM32 FDCAN peripheral
```

Viene di seguito impostata la flag di FreeRTOS `FLAG_CAN_RECEIVE` all'interno del callback di ricezione.

```
1 void HAL_FDCAN_RxFifo0Callback(FDCAN_HandleTypeDef* hfdcan, uint32_t RxFifo0ITs)
2 {
3     if (hfdcan == &hfdcan1) {
4         osEventFlagsSet(rxCANTaskHandle, FLAG_CAN_RECEIVE);
5     }
6 }
```

La flag viene poi usata all'interno di `receiveCANFunc`, prima di fornire il frame CAN ricevuto alla funzione `feedRxFrame`, la quale si occuperà successivamente di inviare il frame in ricezione tramite `libcanard`.

```

1 void receiveCANFunc(void *argument)
2 {
3     for(;;)
4     {
5         osEventFlagsWait(rxCANTaskHandle, FLAG_CAN_RECEIVE, osFlagsWaitAny,
6             osWaitForever);
7
8         while (HAL_FDCAN_GetRxFifoFillLevel(&hfdcan1, FDCAN_RX_FIFO0) > 0)
9         {
10             if (HAL_FDCAN_GetRxMessage(&hfdcan1, FDCAN_RX_FIFO0, &RxHeader, RxData) ==
11                 HAL_OK)
12             {
13                 feedRxFrame(&canard, &RxHeader, RxData);
14             }
15         }
16
17         // Tell the TX thread to immediately shoot out any queued replies
18         osEventFlagsSet(txCANTaskHandle, FLAG_CAN_TRANSMIT);
19     }
20
21     osEventFlagsSet(txCANTaskHandle, FLAG_CAN_TRANSMIT);
22 }

```

```

1 void feedRxFrame(CanardInstance* ins, const FDCAN_RxHeaderTypeDef* rxHdr, const
2     uint8_t* data)
3 {
4     CanardCANFrame frame;
5     frame.id = rxHdr->Identifier | CANARD_CAN_FRAME_EFF;
6     switch (rxHdr->DataLength) {
7         case FDCAN_DLC_BYTES_0: frame.data_len = 0; break;
8         case FDCAN_DLC_BYTES_1: frame.data_len = 1; break;
9         case FDCAN_DLC_BYTES_2: frame.data_len = 2; break;
10        case FDCAN_DLC_BYTES_3: frame.data_len = 3; break;
11        case FDCAN_DLC_BYTES_4: frame.data_len = 4; break;
12        case FDCAN_DLC_BYTES_5: frame.data_len = 5; break;
13        case FDCAN_DLC_BYTES_6: frame.data_len = 6; break;
14        case FDCAN_DLC_BYTES_7: frame.data_len = 7; break;
15        case FDCAN_DLC_BYTES_8: frame.data_len = 8; break;
16        default: frame.data_len = (rxHdr->DataLength <= 8) ? (uint8_t)rxHdr->
17            DataLength : 8; break;
18    }
19
20    for (uint8_t i = 0; i < frame.data_len; i++) {
21        frame.data[i] = data[i];
22    }
23    frame.iface_id = 0;
24
25    uint64_t timestamp_usec = HAL_GetTick() * 1000ULL;
26    int8_t res = canardHandleRxFrame(ins, &frame, timestamp_usec);
27 }

```

La funzione `receiveCANFunc` attiverà poi anche la flag `FLAG_CAN_TRANSMIT`, la quale consentirà alla funzione `sendCANFunc` di codificare e di mandare i dati in trasmissione.

```

1 void sendCANFunc(void *argument)
2 {
3     for(;;)
4     {
5         uint32_t flags = osEventFlagsWait(txCANTaskHandle, FLAG_CAN_TRANSMIT |
6             FLAG_CAN_PERIODIC, osFlagsWaitAny, osWaitForever);
7
8         if(flags & FLAG_CAN_PERIODIC) {
9
10             #if ENABLE_BATTERY                #if ENABLE_MPPT
11                 battery_update();             mppt_update();
12                 send_BatteryInfo();           send_MPPT_Data();
13             #endif                            #endif
14
15             node_status_send();
16         }
17
18         // Emptying the queue
19         if (!processTxQueue(&canard))
20         {
21             // Call the TX task again so it loops back to finish the rest
22             osEventFlagsSet(txCANTaskHandle, FLAG_CAN_TRANSMIT);
23
24             // Give it 1 millisecond to push to the bus
25             osDelay(1);
26         }
27     }
28 }

```

Per essere più specifici, la trasmissione avviene tramite `processTxQueue`:

```

1 bool processTxQueue(CanardInstance* ins)
2 {
3     const CanardCANFrame* txf;
4     while ((txf = canardPeekTxQueue(ins)) != NULL)
5     {
6         FDCAN_TxHeaderTypeDef hdr = {
7             // [...] Generic header definition
8         };
9
10        // TX Length switch
11        switch (txf->data_len) {
12            case 0:  hdr.DataLength = FDCAN_DLC_BYTES_0; break;
13            // [...]
14            default: hdr.DataLength = FDCAN_DLC_BYTES_8; break;
15        }
16
17        HAL_StatusTypeDef status = HAL_FDCAN_AddMessageToTxFifoQ(...);
18
19        if (status == HAL_OK) {
20            canardPopTxQueue(ins);
21        } else {
22            return false; // hardware full
23        }
24    }
25
26    return true;
27 }

```

La funzione `canardHandleRxFrame` chiamata dentro `feedRxFrame` passa il controllo a `libcanard`, dove infatti viene chiamata, da `canlib.c`, la funzione `shouldAcceptTransfer`, grazie alla quale viene iniziato l'assemblaggio dei pacchetti di `NodeInfo`, fondamentale per la corretta identificazione del nodo da parte del Flight Controller, e di `ArmingStatus`, cruciale al fine di consentire alla Nucleo Board di riconoscere lo stato di armamento del Flight Controller.

```

1 bool shouldAcceptTransfer(const CanardInstance* ins,
2                           uint64_t* out_data_type_signature,
3                           uint16_t data_type_id,
4                           CanardTransferType transfer_type,
5                           uint8_t source_node_id)
6 {
7     if(transfer_type == CanardTransferTypeRequest)
8     {
9         // NodeInfo
10        if (data_type_id == UAVCAN_PROTOCOL_GETNODEINFO_ID)
11        {
12            *out_data_type_signature = UAVCAN_PROTOCOL_GETNODEINFO_SIGNATURE;
13            return true;
14        }
15    }
16
17    if(transfer_type == CanardTransferTypeBroadcast)
18    {
19        // MPPT Arming/Disarming
20        if(data_type_id == UAVCAN_EQUIPMENT_SAFETY_ARMINGSTATUS_ID)
21        {
22            *out_data_type_signature =
23                UAVCAN_EQUIPMENT_SAFETY_ARMINGSTATUS_SIGNATURE;
24            return true;
25        }
26    }
27    return false;
28 }

```

In seguito viene anche chiamata la funzione `onTransferReceived`, la quale parte dai pacchetti assemblati e sulla base del contenuto esegue diverse azioni, sia di comunicazione verso il Flight Controller che di analisi dei dati ricevuti da quest'ultimo.

```

1 void onTransferReceived(CanardInstance* ins, CanardRxTransfer* transfer)
2 {
3     if (transfer->transfer_type == CanardTransferTypeRequest)
4     {
5         if (transfer->data_type_id == UAVCAN_PROTOCOL_GETNODEINFO_ID)
6         {
7             handle_GetNodeInfo(ins, transfer);
8         }
9     }
10    else if (transfer->transfer_type == CanardTransferTypeBroadcast)
11    {
12        if (transfer->data_type_id == UAVCAN_EQUIPMENT_SAFETY_ARMINGSTATUS_ID)
13        {
14            handle_ArmingStatus(ins, transfer);
15        }
16    }
17 }

```

A prescindere dal dispositivo selezionato (MPPT e/o PDU) i pacchetti obbligatori per l'adeguato riconoscimento del nodo (qui chiamato `MyDroneNode`) all'interno di Mission Planner sono `NodeStatus` e `GetNodeInfo`, inoltrati con frequenza pari a 1 Hz e gestiti in `node.c`, rispettivamente dalle funzioni `node_status_send` e `handle_GetNodeInfo`, riportate qui di seguito.

[illegible]

3.4.4 La gestione dei dati della PDU

Per gestire i dati della PDU (nel file `pdu.c`) è stata impiegata la funzione `battery_update`, il cui scopo è di fornire dei nuovi valori alla batteria (in questo caso simulati casualmente per fini dimostrativi), mentre per la comunicazione con il Flight Controller è stato impiegata la funzione `send_BatteryInfo`.

```

1 static struct uavcan_equipment_power_BatteryInfo battery_state = {0};
2
3 void battery_update()
4 {
5     // "battery_voltage" in Mission Planner
6     battery_state.voltage = 20 + (rand() % (2 * 5 + 1) - 5);
7
8     // "current" in Mission Planner
9     battery_state.current = 30 + (rand() % (2 * 5 + 1) - 5);
10
11     // "battery_temp" in Mission Planner
12     battery_state.temperature = 25 + (rand() % (2 * 5 + 1) - 5) + 273.15;
13
14     // "battery_remaining" in Mission Planner
15     battery_state.state_of_charge_pct = 60 + (rand() % (2 * 5 + 1) - 5);
16
17     // Name of the battery model, only seen in Mission Planner's Inspector
18     const char* name = "RA_Pack";
19     battery_state.model_name.len = strlen(name);
20     memcpy(battery_state.model_name.data, name, battery_state.model_name.len);
21 }

```

```

1 void send_BatteryInfo(void)
2 {
3     uint8_t buffer[UAVCAN_EQUIPMENT_POWER_BATTERYINFO_MAX_SIZE];
4
5     uint32_t len = uavcan_equipment_power_BatteryInfo_encode(&battery_state,
6         buffer);
7
8     static uint8_t battery_transfer_id;
9
10    canardBroadcast(&canard,
11        UAVCAN_EQUIPMENT_POWER_BATTERYINFO_SIGNATURE,
12        UAVCAN_EQUIPMENT_POWER_BATTERYINFO_ID,
13        &battery_transfer_id,
14        CANARD_TRANSFER_PRIORITY_HIGHEST,
15        buffer,
16        len);
17 }

```

Con un paio di note a margine:

- I campi qui mostrati appariranno su Mission Planner come identici ai valori mostrati, con unica eccezione della temperatura, la quale verrà ridotta di una cifra pari a 273.15 rispetto al valore inserito, di fatto eseguendo automaticamente la conversione da Kelvin a gradi centigradi.
- I campi all'interno del tipo predefinito `uavcan_equipment_power_BatteryInfo` sono in realtà molti di più rispetto agli effettivamente utilizzati, ma quelli mostrati sono gli unici di cui si è dimostrata una corrispondenza effettiva all'interno dell'interfaccia grafica di Mission Planner, dove invece gli altri sono stati correttamente ricevuti, ma senza un riscontro pratico reale all'interno del software.

3.4.5 La gestione dei dati dell'MPPT

Per gestire i dati dell'MPPT (nel file `mppt.c`) è stata impiegata la funzione `mppt_update`, il cui scopo è quello di fornire dei nuovi valori al dispositivo (in questo caso simulati casualmente per fini dimostrativi). Per la gestione in risposta allo stato di armamento è stata usata la funzione `handle_ArmingStatus`, mentre per la comunicazione con l'autopilota è stata impiegata la funzione `send_MPPT_Data`.

```

1 static struct mppt_Data mppt_state = {0};
2
3 void mppt_update(void)
4 {
5     // "esc15_volt" in Mission Planner --> "MPPT_In_Voltage (V)"
6     mppt_state.input_voltage = 75 + (rand() % 10);
7
8     // "esc15_curr" in Mission Planner --> "MPPT_In_Current (A)"
9     mppt_state.input_current = 10 + (rand() % 5);
10
11    // "esc15_rpm" in Mission Planner --> "MPPT_In_Power (W)"
12    mppt_state.input_power = 600 + (rand() % 50);
13
14    // "esc16_volt" in Mission Planner --> "MPPT_Out_Voltage (V)"
15    mppt_state.output_voltage = 25 + (rand() % 5);
16
17    // "esc16_curr" in Mission Planner --> "MPPT_Out_Current (A)"
18    mppt_state.output_current = 35 + (rand() % 5);
19
20    // "esc16_rpm" in Mission Planner --> "MPPT_Out_Power (W)"
21    mppt_state.output_power = 200 + (rand() % 20);
22
23    // "esc15_temp" in Mission Planner --> "MPPT_Temperature (C)"
24    mppt_state.temperature = 30 + (rand() % 5) + 273.15;
25 }

```

```

1 void handle_ArmingStatus(CanardInstance* ins, CanardRxTransfer* transfer)
2 {
3     struct uavcan_equipment_safety_ArmingStatus arm;
4     if (uavcan_equipment_safety_ArmingStatus_decode(transfer, &arm)) {
5         return;
6     }
7
8     if (arm.status > 0) {
9         HAL_GPIO_WritePin(LD2_GPIO_Port, LD2_Pin, GPIO_PIN_SET);
10    } else {
11        HAL_GPIO_WritePin(LD2_GPIO_Port, LD2_Pin, GPIO_PIN_RESET);
12    }
13 }

```

Con un paio di note a margine:

- I campi qui mostrati appariranno su Mission Planner come identici ai valori mostrati, con unica eccezione della temperatura, la quale verrà ridotta di una cifra pari a 273.15 rispetto al valore inserito, di fatto eseguendo automaticamente la conversione da Kelvin a gradi centigradi.
- I campi all'interno del tipo predefinito `uavcan_equipment_esc.Status`, come mostrato qui di seguito in `send_MPPT_Data`, sono in realtà molti di più rispetto a quelli effettivamente mostrati, in quanto gli unici fondamentali per la gestione della telemetria relativa alla strumentazione ritenuta di valore per questo progetto.

- Essendo i pacchetti per l'MPPT relativi sia all'ingresso che all'uscita di tensione, corrente e potenza, e non essendoci abbastanza campi per soddisfare con un solo pacchetto tutte le richieste, sono stati previsti un totale di due messaggi, come visibile qui di seguito, dove il primo è utilizzato per contenere i dati di input, mentre il secondo per la memorizzazione dei dati di output.

```

1 void send_MPPT_Data(void)
2 {
3     static uint8_t esc_tid;
4     uint8_t buffer[UAVCAN_EQUIPMENT_ESC_STATUS_MAX_SIZE];
5     struct uavcan_equipment_esc_Status msg;
6
7     // MESSAGE 1: INPUT DATA (Pretending to be ESC Index 15)
8     memset(&msg, 0, sizeof(msg));
9     msg.esc_index = 14;
10    msg.voltage      = mppt_state.input_voltage;
11    msg.current      = mppt_state.input_current;
12    msg.temperature  = mppt_state.temperature;
13    msg.rpm          = mppt_state.input_power;
14    msg.error_count  = 0;
15
16    uint32_t len = uavcan_equipment_esc_Status_encode(&msg, buffer);
17    canardBroadcast(&canard,
18                    UAVCAN_EQUIPMENT_ESC_STATUS_SIGNATURE,
19                    UAVCAN_EQUIPMENT_ESC_STATUS_ID,
20                    &esc_tid,
21                    CANARD_TRANSFER_PRIORITY_LOW,
22                    buffer, len);
23
24    // MESSAGE 2: OUTPUT DATA (Pretending to be ESC Index 16)
25    memset(&msg, 0, sizeof(msg));
26    msg.esc_index = 15;
27    msg.voltage      = mppt_state.output_voltage;
28    msg.current      = mppt_state.output_current;
29    msg.temperature  = 0; // Not needed
30    msg.rpm          = mppt_state.output_power;
31    msg.error_count  = 0;
32
33    len = uavcan_equipment_esc_Status_encode(&msg, buffer);
34    canardBroadcast(&canard,
35                    UAVCAN_EQUIPMENT_ESC_STATUS_SIGNATURE,
36                    UAVCAN_EQUIPMENT_ESC_STATUS_ID,
37                    &esc_tid,
38                    CANARD_TRANSFER_PRIORITY_LOW,
39                    buffer, len);
40 }

```

3.4.6 Lo script Python per la visualizzazione dell'MPPT su Mission Planner

Come si è visto nella sezione precedente, per l'invio dei pacchetti relativi all'MPPT non è stata usata una struttura predefinita del tipo `MPPT_Stream`, ma una "impropria", del tipo `uavcan_equipment_esc.Status`, in quanto la prima, essendo considerata da Mission Planner come "Vendor Specific", avrebbe richiesto la modifica diretta del codice interno di ArduPilot e dei suoi campi DSDL. Per evitare di passare a un firmware custom, si è cercato un pacchetto il più possibile equivalente nei suoi campi a `MPPT_Stream`, così da preservare la generale struttura dei dati, tenendo allo stesso tempo la struttura del codice grossomodo invariata, in un compromesso tra comodità di risoluzione e mantenibilità della repository.

La scelta finale è ricaduta su `uavcan_equipment_esc.Status` per via dei suoi campi di corrente, tensione, potenza e temperatura, del tutto simili a quelli dell'MPPT, oltre che per via della facilità con cui più pacchetti possono all'occorrenza essere spostati su diversi campi, facilitando notevolmente la gestione dei dati tramite Mission Planner.

Per motivi prettamente estetici, si è poi deciso di scrivere uno script Python interno a Mission Planner stesso (e quindi non gravante sulla capacità di calcolo del Flight Controller e/o della Nucleo Board) per la gestione dei dati, così da spostare i valori dell'ESC in dei custom field appositi, poi rinominati per accomodare le reali misure mostrate.

Lo script è il seguente:

```

1 import time
2
3 cs.custom_field_names["customfield0"] = "MPPT_In_Voltage (V)"
4 cs.custom_field_names["customfield1"] = "MPPT_In_Current (A)"
5 cs.custom_field_names["customfield2"] = "MPPT_In_Power (W)"
6 cs.custom_field_names["customfield3"] = "MPPT_Temperature (C)"
7 cs.custom_field_names["customfield4"] = "MPPT_Out_Voltage (V)"
8 cs.custom_field_names["customfield5"] = "MPPT_Out_Current (A)"
9 cs.custom_field_names["customfield6"] = "MPPT_Out_Power (W)"
10
11 while True:
12     try:
13         cs.customfield0 = float(cs.esc15_volt)
14         cs.customfield1 = float(cs.esc15_curr)
15         cs.customfield2 = float(cs.esc15_rpm)
16         cs.customfield3 = float(cs.esc15_temp)
17         cs.customfield4 = float(cs.esc16_volt)
18         cs.customfield5 = float(cs.esc16_curr)
19         cs.customfield6 = float(cs.esc16_rpm)
20     except:
21         pass
22     Script.Sleep(200)

```

Ed è possibile inserirlo tramite Mission Planner al seguente percorso:

Data → Scripts → Select Script

Per poi cliccare su **Run Script** e selezionare i nuovi campi appena creati nella sezione **Quick**.

NB: Gli ESC scelti sono stati il numero 15 e il numero 16 perché, essendo questi gli ultimi della lista, essi danno la garanzia di essere dei campi che non verranno mai utilizzati da altri dispositivi per degli scopi associabili, così quindi da consentire un'alta mantenibilità del codice.

3.5 La visualizzazione finale

Una volta eseguito lo script Python e configurata la sezione **Quick** di Mission Planner con i campi di interesse, la visuale dell'intero blocco di pacchetti presenterà la seguente forma:

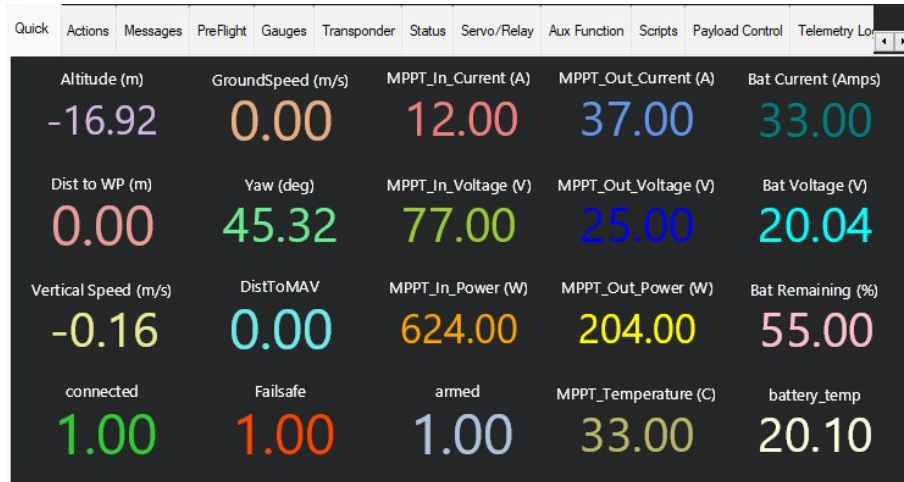


Figura 3.7: Visualizzazione finale dei pacchetti di Flight Controller, PDU e MPPT

```

ID 127 - org.missionplanner ~0Bps
  -uavcan_protocol_GetNodeInfo_req (0.0 Hz, #1) ~0Bps
  +uavcan_protocol_NodeStatus (1.0 Hz, #341) ~0Bps
ID 20 - org.ardupilot:0 ~307Bps
  +ardupilot_indication_NotifyState (24.7 Hz, #20007) ~296Bps
  +ardupilot_indication_SafetyState (2.0 Hz, #20000) ~2Bps
  +uavcan_equipment_safety_ArmingStatus (2.0 Hz, #1100) ~2Bps
  +uavcan_protocol_GetNodeInfo_req (0.0 Hz, #1) ~0Bps
  +uavcan_protocol_NodeStatus (1.0 Hz, #341) ~7Bps
ID 31 - MyDroneNode ~1603Bps
  +uavcan_equipment_esc_Status (49.3 Hz, #1034) ~691Bps
    -current 36 Single
    -error_count 0 UInt32
    -esc_index 15 Byte
    -power_rating_pct 0 Byte
    -rpm 218 Int32
    -temperature 0 Single
    -voltage 28 Single
  +uavcan_equipment_power_BatteryInfo (24.7 Hz, #1092) ~740Bps
    -average_power_10sec 0 Single
    -battery_id 0 Byte
    -current 31 Single
    -full_charge_capacity_wh 0 Single
    -hours_to_full_charge 0 Single
    -model_instance_id 0 UInt32
    -model_name 82,65,95,80,97,99,107 Byte[]
    -model_name_len 7 Byte
    -remaining_capacity_wh 0 Single
    -state_of_charge_pct 62 Byte
    -state_of_charge_pct_stdev 0 Byte
    -state_of_health_pct 0 Byte
    -status_flags 0 UInt16
    -temperature 300.25 Single
    -voltage 24 Single
  +uavcan_protocol_GetNodeInfo_res (0.0 Hz, #1) ~0Bps
  +uavcan_protocol_NodeStatus (24.7 Hz, #341) ~173Bps

```

Figura 3.8: Visualizzazione finale dei dispositivi tramite l'UAVCAN Inspector di Mission Planner

CAPITOLO 4

Conclusione

4.1 Resoconto dell'esperienza

Penso che questo progetto sia stato per me enormemente formativo, sia per l'esperienza sul campo che per le grandi conoscenze acquisite lungo il percorso. Lo studio del firmware e dei sistemi embedded presenta delle sfide talvolta di difficile risoluzione, spesso estranee a qualsiasi argomento trattato a lezione e dunque necessitanti di molto spirito di iniziativa, tenacia e pensiero logico di approccio ai problemi.

Trovo che lavorare con Icarus mi abbia dato una grande mano per quanto riguarda la scelta del mio percorso futuro, sia per la scelta ormai compiuta della laurea magistrale che per una sempre più vicina carriera nel mondo del lavoro. Grazie al team studentesco ho avuto la possibilità di conciliare due delle mie più grandi passioni, quali l'informatica e l'aerospazio, in un ambiente che difficilmente sarei riuscito a trovare altrove.

Uno dei problemi più grandi all'interno di gruppi studenteschi di questo tipo è il continuo succedersi di membri sempre nuovi, i quali difficilmente, per via del termine degli studi, rimarranno tra le fila del team per più di qualche anno. Questo continuo turnover, pur apportando nuove energie e entusiasmo al team, rende tuttavia difficile tramandare le conoscenze acquisite nel tempo. Relazioni come questa si pongono come obiettivo principale quello di porre soluzione proprio a questo genere di difficoltà, catalogando il know-how e rendendolo disponibile a chiunque siglerà il futuro successo del team.

Così come altri hanno avuto la premura di insegnarmi e di rendermi ciò che sono, io stesso proverò al meglio delle mie possibilità a portare altri sotto la mia ala, preferibilmente a propulsori spenti, per consentirgli di imparare a camminare, a correre, e un giorno perfino a volare.

Grazie per l'attenzione.

4.2 Ringraziamenti

Oltre a tutti i membri di Icarus, senza i quali niente di questo sarebbe stato possibile, vorrei dedicare questo passaggio finale ad alcune persone in particolare.

In primis, ringrazio Ed e Mattia di AESA (oltre che già membri di Icarus) per avermi spinto ad entrare nel team, decisione che è senza alcun dubbio valsa il tempo e le energie dedicateci.

In secondo luogo, ringrazio Angelo e Samuele per l'impegno e la dedizione dimostrati nei miei confronti e verso gli altri membri del team, per averci trasmesso tutto ciò che sappiamo sui sistemi embedded e per le numerose mattinate e serate trascorse insieme al Toolbox.

And lastly, I'd like to thank Matteo and Burak for helping me throughout the whole project, from the coding, the debugging, the troubleshooting, all the way to the endless brainstorming over Mission Planner's darkest corners and the various sub-modules used to be able to have the final work up and running. Memories were made.